Birla Vishwakarma Mahavidhyalaya

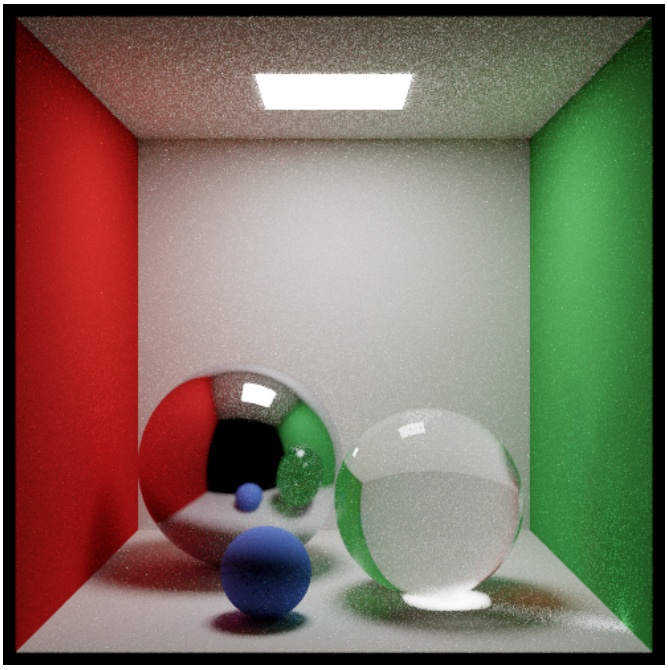
(An Autonomous Institution)

C++ PROJECT REPORT – OBJECT ORIENTED PROGRAMMING

Software-Based Ray Tracer

A High-Performance 3D Rendering Engine

with Global Illumination, BVH Acceleration
and Multi-Core Parallelization



Cornell box rendered by the engine described in this report

Name: Vinit Nitinbhai Dubal
ID No.: 25EC413
Course Code: 204ES
Subject: Object Oriented Programming (C++)
Date: October 24, 2026

Abstract

Ray tracing is the family of rendering algorithms that produces images by simulating how light travels through a scene. This report documents the design, implementation and evaluation of a standalone software ray tracer written in modern C++ with no external libraries. The renderer is a Monte Carlo *path tracer*: it computes global illumination effects such as colour bleeding, soft area-light shadows, mirror reflections and glass refraction. It is accelerated by a Bounding Volume Hierarchy (BVH) built with a binned Surface Area Heuristic (SAH) and traversed iteratively front-to-back. Further quality and speed features include next-event estimation (direct light sampling), adaptive per-pixel sampling, Russian roulette termination, a thin-lens depth-of-field camera, ACES filmic tone mapping with sRGB encoding, a tile-based multi-threaded scheduler with per-tile random seeding, and a self-contained PNG writer. All figures and measurements in this report were produced by running the code itself: on the 278-object sphere scene the BVH delivers a **5.3**× reduction in render time over brute-force intersection, and renders are bit-identical for any thread count.

Keywords: ray tracing, path tracing, global illumination, BVH, surface area heuristic, multithreading, C++.

Contents

Abstract i

1 Introduction 1

2 Problem Statement and Scope 1

3 Technology and Tools Used 2

4 Theoretical Background 2

5 System Architecture and Object-Oriented Design 3

6 Implementation Details 4

7 Test Scenes and Rendered Output 7

8 Experiments and Performance Results 8

9 Testing and Validation 10

10 Limitations and Future Work 10

11 Conclusion 11

References 11

A Command-Line Interface 11

1 Introduction

1.1 Background

Computer graphics converts a mathematical description of a 3D scene (geometry, materials, lights and a camera) into a 2D image. Real-time engines usually rely on *rasterization*, which projects triangles onto the screen and approximates lighting with local shading models. *Ray tracing* takes the opposite route: for every pixel it shoots rays from the camera into the scene and follows them as they bounce, which makes physically motivated effects such as reflections, refractions and indirect illumination emerge naturally instead of being faked with special-purpose tricks. The price of this generality is computational cost, since a single image may require hundreds of millions of ray–scene intersection tests.

1.2 Motivation

A ray tracer is an excellent vehicle for studying Object Oriented Programming and systems-level C++: it needs abstract data types with overloaded operators (vectors), value semantics, composition of small cooperating components (materials, primitives, lights, acceleration structures), the STL, lambdas, and safe concurrency. It also makes the effect of algorithmic choices immediately visible, both in the picture (noise, aliasing, shadows) and in the performance numbers (rays per second).

1.3 Objectives

1. Implement a standalone C++ renderer that produces an image file without any third-party library.
2. Support spheres and triangles, and the three fundamental material classes: diffuse, metallic and dielectric (glass), plus emissive area lights.
3. Simulate global illumination using Monte Carlo path tracing.
4. Accelerate ray–scene intersection with a BVH built using the surface area heuristic.
5. Exploit all CPU cores through multi-threaded tile rendering.
6. Present measured outputs and performance results that validate each feature.

1.4 Organisation of the Report

Section 2 states the problem and requirements, Section 3 lists the technology used, and Section 4 reviews the underlying theory. Section 5 presents the architecture and object-oriented design, Section 6 walks through the implementation, Sections 7 and 8 show the rendered output and experiments, and Sections 10–11 discuss limitations, future work and conclusions.

2 Problem Statement and Scope

> *Design and implement a standalone, high-performance C++ software ray tracer capable of rendering complex 3D scenes with global illumination effects, optimized using spatial acceleration structures (BVH) and multi-core thread parallelization.*

The statement is decomposed into measurable requirements, each traced to a feature of the implementation in Table 1.

Table 1: Requirements derived from the problem statement.

ID	Requirement	Realisation in the code
R1	Standalone, no external dependencies	Single `raytracer.cpp`; own PNG encoder; only the C++ standard library
R2	Complex 3D scenes	Spheres + triangles; scenes with 15 and 278 primitives
R3	Global illumination	Path tracing with up to 16 bounces, Russian roulette, colour bleeding, caustics
R4	Realistic materials	Lambertian, metal (roughness), glass (Snell + Schlick), area lights
R5	Spatial acceleration (BVH)	Binned-SAH BVH, iterative front-to-back traversal
R6	Multi-core parallelism	`std::thread` pool, atomic tile queue, thread-local RNG
R7	High performance / efficiency	Next-event estimation, adaptive sampling, `-O3`, rays/s statistics
R8	Visible output	PNG image, console statistics, progress bar

Scope. The engine renders offline (not real-time) on the CPU. Scene description is done in C++ code (`buildScene`);

loading of external mesh files and image textures is out of scope and listed under future work.

3 Technology and Tools Used

The project deliberately uses only the C++ standard library so that it builds with a single command on any platform with a C++11 compiler.

Table 2: Technology stack.

Component	Use in the project
C++11 (`-std=c++11`)	Lambdas, `thread_local`, `auto`, `<atomic>`, `<thread>`, range of STL algorithms
`std::thread`, `std::atomic`	Worker pool and lock-free tile counter (no OpenMP needed)
`<random>` (`mt19937`)	Per-thread Mersenne-Twister for Monte Carlo sampling
`<chrono>`	Timing, progress bar, ETA, Mrays/s
STL (`vector`, `partition`, `nth_element`)	Scene storage and BVH construction
Own PNG writer (CRC-32, Adler-32, stored deflate)	Image export without zlib/libpng
g++ `-O3 -pthread`	Optimising compiler used for all measurements

4 Theoretical Background

4.1 The Rendering Equation and Path Tracing

The radiance leaving a surface point $\mathbf{x}$ in direction ω_o is described by Kajiya's rendering equation:

$$L_o(\mathbf{x},\omega_o) = L_e(\mathbf{x},\omega_o) + \int_\Omega f_r(\mathbf{x},\omega_i,\omega_o)\, L_i(\mathbf{x},\omega_i)\,(\mathbf{n}\cdot\omega_i)\, d\omega_i. \tag{1}$$

The integral is recursive, because the incoming radiance L_i is itself the outgoing radiance of another surface. *Path tracing* estimates it with Monte Carlo integration: a random path is traced from the camera, scattering at each hit according to the material, and the contributions are averaged over many samples per pixel (spp). The estimate is unbiased and its noise decreases as $1/\sqrt{N}$ in the number of samples N.

4.2 Next-Event Estimation

Pure path tracing finds a small light only by chance, which is very noisy. *Next-event estimation* (direct light sampling) adds, at every diffuse hit, an explicit shadow ray towards a random point $\mathbf{p}$ on a light of area A. For a Lambertian surface of albedo ρ the estimator is

$$L_{\text{direct}} \approx \rho\, L_e \,\frac{\cos\theta_s \cos\theta_l\, A}{\pi\,\|\mathbf{p}-\mathbf{x}\|^2}\, V(\mathbf{x},\mathbf{p})\, N_{\text{lights}}, \tag{2}$$

where θ_s and θ_l are the angles at the surface and the light, $V \in \{0,1\}$ is the visibility from the shadow ray, and the factor N_{lights} compensates for choosing one light uniformly. To avoid counting a light twice, light emission found by a ray that came from a diffuse bounce is ignored; emission is added only after the camera ray or a specular (mirror/glass) bounce.

4.3 Russian Roulette

Paths are cut off after a few bounces without bias: at depth > 3 a path survives with probability $p = \min(\max(w_r, w_g, w_b), 0.95)$, where w is the current throughput, and the throughput of survivors is divided by p. Dark, low-contribution paths therefore terminate early while the expected value is preserved.

4.4 Materials

Lambertian (diffuse). The scattered direction is $\mathbf{n}+\xi$ with ξ a random unit vector, which gives a cosine-weighted hemisphere distribution. **Metal.** Mirror reflection $\mathbf{r} = \mathbf{v} - 2(\mathbf{v}\cdot\mathbf{n})\mathbf{n}$ perturbed by a roughness-scaled random vector ("fuzz"). **Dielectric (glass).** With the ratio of refractive indices η, Snell's law $\eta \sin\theta = \sin\theta'$ gives the refracted direction; when $\eta \sin\theta > 1$ total internal reflection occurs. Otherwise reflection is chosen with the Fresnel probability approximated by Schlick's formula,

$$R(\theta) = R_0 + (1-R_0)(1-\cos\theta)^5, \qquad R_0 = \left(\frac{1-\eta}{1+\eta}\right)^2. \tag{3}$$

4.5 Bounding Volume Hierarchy and the Surface Area Heuristic

Testing every ray against all N primitives costs $O(N)$. A BVH wraps groups of primitives in axis-aligned bounding boxes (AABB) organised as a binary tree, so that a ray that misses a box skips everything inside it; the expected cost becomes $O(\log N)$. The quality of the tree depends on how nodes are split. The Surface Area Heuristic estimates the cost of splitting a node with area S_P into children with areas S_L, S_R and primitive counts N_L, N_R as

$$C_{\text{split}} = C_{\text{trav}} + \frac{S_L N_L + S_R N_R}{S_P} C_{\text{isect}}, \tag{4}$$

since the probability that a random ray hitting the parent also hits a child is proportional to the child's surface area. The implementation uses *binned* SAH: centroids are placed into 12 bins along the longest axis, and prefix/suffix sweeps evaluate all 11 candidate splits in linear time.

4.6 Thin-Lens Camera

Rays originate at a random point on a lens disk of radius $a/2$ and aim at the point on the focal plane, at distance d_f, that the pinhole ray through the same pixel would hit. Objects at d_f stay sharp while others blur, producing depth of field. Random sub-pixel jitter of the primary ray provides anti-aliasing.

4.7 Tone Mapping and Colour Encoding

Radiance values are unbounded (the lamp has value 15), so they are compressed with Narkowicz's ACES filmic fit

$$T(x) = \frac{x(2.51x + 0.03)}{x(2.43x + 0.59) + 0.14}, \tag{5}$$

clamped to $[0, 1]$, and then encoded with the sRGB transfer function ($12.92x$ for $x \le 0.0031308$, otherwise $1.055\,x^{1/2.4} - 0.055$).

5 System Architecture and Object-Oriented Design

5.1 Module Overview

The program is organised as a pipeline of cooperating modules (Figure 1), each mapped to a section of the source file.

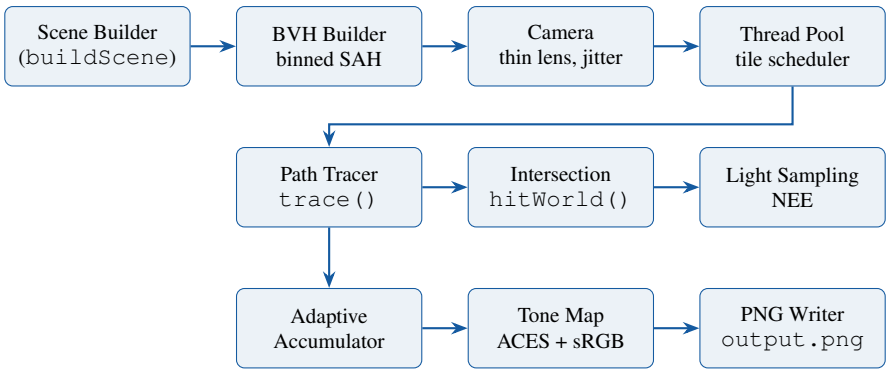


Figure 1: Processing pipeline of the ray tracer.

5.2 Class Structure

The design is built from small, focused user-defined types that are composed together (Figure 2). Each type owns its data and exposes only the operations needed by its clients.

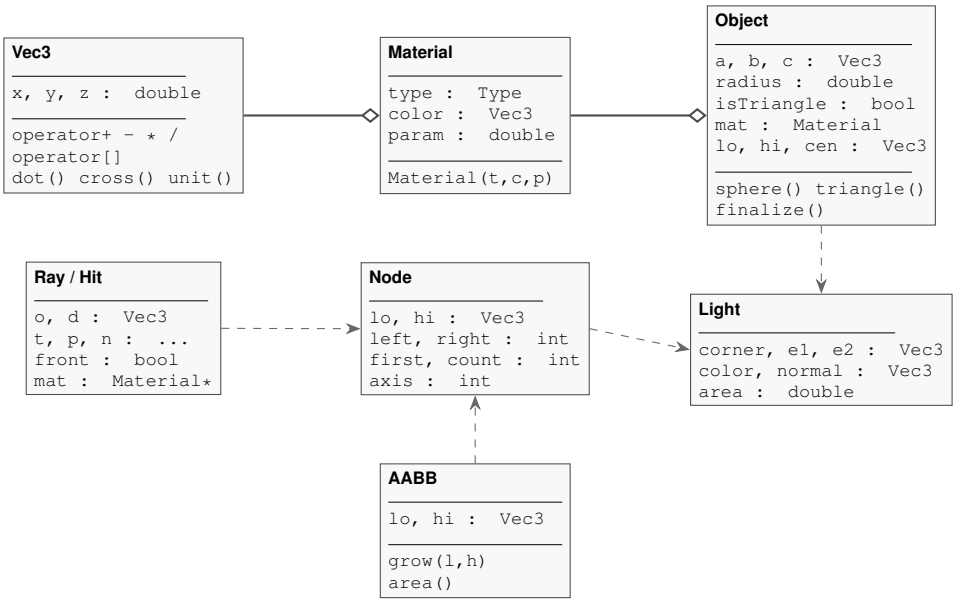


Figure 2: Simplified UML class diagram. Solid diamonds show composition (`Material` and `Object` contain `Vec3`; `Object` contains a `Material`); dashed arrows show "uses" relationships.

5.3 Object-Oriented Concepts Applied

Table 3 maps the OOP principles studied in the course to concrete places in the implementation.

Table 3: OOP and modern-C++ concepts used.

Concept	Where and how it is used
Abstraction / ADTs	`Vec3` models a mathematical 3D vector and is used for points, directions and RGB colours alike
Encapsulation	Related data and behaviour are bundled in `Material`, `Object`, `AABB`, `Node`, `Light`; `AABB` hides how bounds are grown and measured via `grow()` and `area()`
Constructors	Default-argument constructors (`Vec3(x=0,y=0,z=0)`, `Material(t,c,p)`) and `Node()`/`AABB()` that initialise safe defaults (empty box, child index -1)
Operator overloading	`+ - * /`, unary $-$ and `operator[]` on `Vec3` make the maths read like the equations in Section 4
Enumerations	`enum Type {DIFFUSE, METAL, GLASS, LIGHT}` selects the scattering model
Composition	`Object` has a `Material`; `Hit` refers to a `Material`; the scene is a `vector<Object>`
Factory functions	`sphere()`, `triangle()`, `quad()`, `areaLight()` build and register primitives
Lambdas / closures	Tile worker `work`, BVH `binOf`, `makeLeaf` and partition predicates
RAII & STL containers	`vector`, `thread`, `ofstream` release resources automatically; no raw `new`/`delete`
Thread-safety by design	`thread_local` RNG and counters, `atomic` tile index, disjoint writes to the image buffer

Design discussion. Primitives are stored as one flat `Object` struct with an `isTriangle` tag rather than as a polymorphic hierarchy. A virtual `Hittable` base class would be the textbook OOP approach, but it adds a pointer indirection and a virtual call per intersection test and scatters objects in memory, which hurts the BVH's cache behaviour. The flat array lets the BVH reorder primitives in place and visit leaves linearly, so it was chosen for performance; Section 10 describes how a polymorphic interface could be layered on top.

6 Implementation Details

6.1 Math Core

All geometry and colour arithmetic is carried out by `Vec3`. Overloaded operators keep the rest of the code compact and readable.

Listing 1: Vec3 with overloaded operators (excerpt).

```
struct Vec3 {
    double x, y, z;
    Vec3(double x = 0, double y = 0, double z = 0) : x(x), y(y), z(z) {}
```

```
    double operator[](int i) const { return i == 0 ? x : (i == 1 ? y : z); }
};
Vec3 operator+(Vec3 a, Vec3 b) { return Vec3(a.x + b.x, a.y + b.y, a.z + b.z); }
Vec3 operator*(Vec3 a, Vec3 b) { return Vec3(a.x * b.x, a.y * b.y, a.z * b.z); }
Vec3 operator*(double t, Vec3 a) { return Vec3(t * a.x, t * a.y, t * a.z); }
double dot(Vec3 a, Vec3 b)  { return a.x*b.x + a.y*b.y + a.z*b.z; }
Vec3 reflect(Vec3 v, Vec3 n) { return v - 2 * dot(v, n) * n; }
```

Random numbers come from a `thread_local mt19937`, so threads never contend for generator state and never need locks; `randomUnit()` and `randomInDisk()` use rejection sampling.

6.2 Primitives and Intersection

Spheres are intersected by solving the quadratic $|\mathbf{o}+t\mathbf{d}-\mathbf{c}|^2 = r^2$ and taking the nearest root in (ε, t_{max}) with $\varepsilon = 10^{-3}$ to prevent self-intersection ("shadow acne"). Triangles use the Möller–Trumbore algorithm, which computes the barycentric coordinates (u, v) and the distance t with two cross products and no precomputed plane equation. Both fill the same `Hit` record, flipping the normal so that it always opposes the incoming ray and recording whether the hit was on the front face, which the glass material needs to know whether the ray enters or leaves the medium. A bounding box and centroid for every primitive are computed once by `finalize()`; flat triangles are padded by 10^{-4} so their boxes have non-zero thickness.

6.3 BVH Construction

`build()` is a recursive top-down builder over a range of the `objects` array. At each node it bounds the primitives, selects the axis with the largest centroid extent, and evaluates the binned SAH. Leaves are created when the node has at most two primitives, or when splitting is not cheaper than intersecting (at most four primitives). If SAH cannot separate the primitives (e.g. coincident centroids) the builder falls back to a median split with `nth_element`, which guarantees termination.

Listing 2: Binned SAH sweep (excerpt from `build()`).

```
const int B = 12;                       // number of bins
AABB bb[B]; int cnt[B] = {0};
for (int i = start; i < end; i++) {     // 1. bin centroids
    int b = binOf(objects[i]);
    cnt[b]++; bb[b].grow(objects[i].lo, objects[i].hi);
}
double rightCost[B]; AABB acc; int c = 0;
for (int i = B - 1; i > 0; i--) {       // 2. suffix sweep: right-side cost
    acc.grow(bb[i].lo, bb[i].hi); c += cnt[i];
    rightCost[i] = c ? acc.area() * c : 1e30;
}
AABB lacc; int lc = 0; double best = 1e30; int bestSplit = -1;
for (int i = 0; i < B - 1; i++) {       // 3. prefix sweep: pick cheapest split
    lacc.grow(bb[i].lo, bb[i].hi); lc += cnt[i];
    if (lc == 0 || lc == n) continue;
    double cost = lacc.area() * lc + rightCost[i + 1];
    if (cost < best) { best = cost; bestSplit = i; }
}
```

6.4 BVH Traversal

Traversal is iterative with a small explicit stack of 64 node indices, avoiding recursion overhead. The ray's reciprocal direction is computed once and reused by the slab test. Children are pushed so that the child on the near side of the split axis is popped first; because `tmax` shrinks as hits are found, the far child is then often rejected by its box test. For shadow rays an `anyHit` flag returns as soon as any blocker is found.

Listing 3: Iterative front-to-back BVH traversal.

```
bool hitBVH(const Ray &r, double &tmax, Hit &h, bool anyHit) {
    Vec3 inv(1 / r.d.x, 1 / r.d.y, 1 / r.d.z);
    int stack[64], sp = 0;  stack[sp++] = 0;  bool hit = false;
    while (sp) {
        const Node &n = nodes[stack[--sp]];
        if (!hitBox(n, r, inv, tmax)) continue;       // slab test
        if (n.count > 0) {                            // leaf
            for (int i = n.first; i < n.first + n.count; i++)
                if (hitObject(objects[i], r, tmax, h)) {
                    hit = true; tmax = h.t;
                    if (anyHit) return true;          // shadow ray: stop early
                }
        } else if (r.d[n.axis] < 0) {                 // visit nearer child first
            stack[sp++] = n.left;  stack[sp++] = n.right;
        } else {
            stack[sp++] = n.right; stack[sp++] = n.left;
        }
```

```
    }
    return hit;
}
```

6.5 Path Tracer

`trace()` is an iterative loop of up to 16 bounces. It maintains the accumulated `color`, the path `weight` (throughput) and a flag `lastSpecular`. On a miss it adds the sky gradient (scene 1) or nothing (the closed Cornell box). On a diffuse hit it adds direct light via `sampleLights()` and then continues with a cosine-distributed bounce; metals reflect with fuzz; glass refracts or reflects using Schlick probability. Russian roulette is applied after the fourth bounce.

Listing 4: Core of the path tracing loop (condensed).

```
for (int depth = 0; depth < 16; depth++) {
    Hit h;
    if (!hitWorld(ray, h)) { /* add sky colour if enabled */ break; }
    const Material &m = *h.mat;
    if (m.type == LIGHT) {                      // emission, counted once
        if (lastSpecular || lights.empty()) color = color + weight * m.color;
        break;
    }
    if (m.type == DIFFUSE) {
        color = color + weight * m.color * sampleLights(h);   // next-event estimation
        dir = h.n + randomUnit();  lastSpecular = false;
    } else if (m.type == METAL) {
        dir = reflect(unit(ray.d), h.n) + m.param * randomUnit();
        lastSpecular = true;
    } else { /* GLASS: Snell refraction or Schlick reflection */ }
    if (m.type != GLASS) weight = weight * m.color;
    ray.o = h.p;  ray.d = dir;
    if (depth > 3) {                            // Russian roulette
        double p = min(max(weight.x, max(weight.y, weight.z)), 0.95);
        if (rnd() > p) break;
        weight = weight / p;
    }
}
```

6.6 Camera and Adaptive Sampling

For every pixel the renderer fires batches of 16 jittered, lens-sampled primary rays. Each sample's luminance is accumulated into $\sum L$ and $\sum L^2$; after at least 16 samples the pixel's standard error $\sqrt{\mathrm{Var}/n}$ is compared with $\tau\,(0.05 + \bar{L})$, where τ is the user-supplied noise threshold (default 0.03). If the pixel is already below it, sampling stops early, so flat walls and sky use few samples while noisy regions (caustics, glass, penumbrae) receive the full budget. Rare "firefly" samples are clamped to a maximum component of 100.

6.7 Multi-Core Tile Scheduler

The image is split into 16×16 tiles. Worker threads obtain tiles from a shared `atomic<int>` counter, which gives dynamic load balancing without locks: a thread that finishes an easy tile simply takes the next one. At the start of each tile the thread-local generator is re-seeded with the tile index, which makes the output *independent of the thread count and scheduling order*; this property is verified in Section 8. Each thread writes only to the pixels of its own tile, so the image buffer needs no synchronisation, and per-thread ray counters are added to atomic totals when a worker exits. The main thread prints a progress bar with an ETA.

Listing 5: Lock-free tile scheduling (condensed).

```
atomic<int> nextTile(0), doneTiles(0);
auto work = [&]() {
    for (int t; (t = nextTile++) < numTiles;) {
        rng.seed(t + 1);                        // per-tile seed => deterministic image
        /* for each pixel in tile: adaptive sampling loop -> image[y*width+x] */
        doneTiles++;
    }
    totalRays += rayCount;  totalSamples += samples;
};
vector<thread> pool;
for (int i = 0; i < numThreads; i++) pool.push_back(thread(work));
```

6.8 Image Output

Because no compression library is allowed, `savePNG()` writes a valid PNG manually: it prepends a filter byte to every scanline, wraps the data in a zlib stream made of uncompressed ("stored") deflate blocks of up to 65 535 bytes, computes the Adler-32 checksum, and emits the `IHDR`, `IDAT` and `IEND` chunks each protected by a CRC-32. The result opens in any viewer.

7 Test Scenes and Rendered Output

Two scenes are built in `buildScene()` and selected on the command line.

Scene 1 – Random spheres (278 primitives). A large ground sphere carries a randomly generated field of small spheres, 65% diffuse, 20% metal and 15% glass, with three large hero spheres (glass, diffuse brown, polished metal). It is lit by a sky gradient and rendered with a thin-lens camera (aperture 0.08), so the focus plane lies on the hero spheres.

Scene 2 – Cornell box (15 primitives). Five walls (made from 10 triangles; the green wall at $x = 0$ and the red wall at $x = 10$, which the camera sees on the right and left of the image respectively), a rectangular area light in the ceiling made of two emissive triangles, and three spheres: glass, polished metal (roughness 0.05) and blue diffuse. This is the standard test for global illumination.

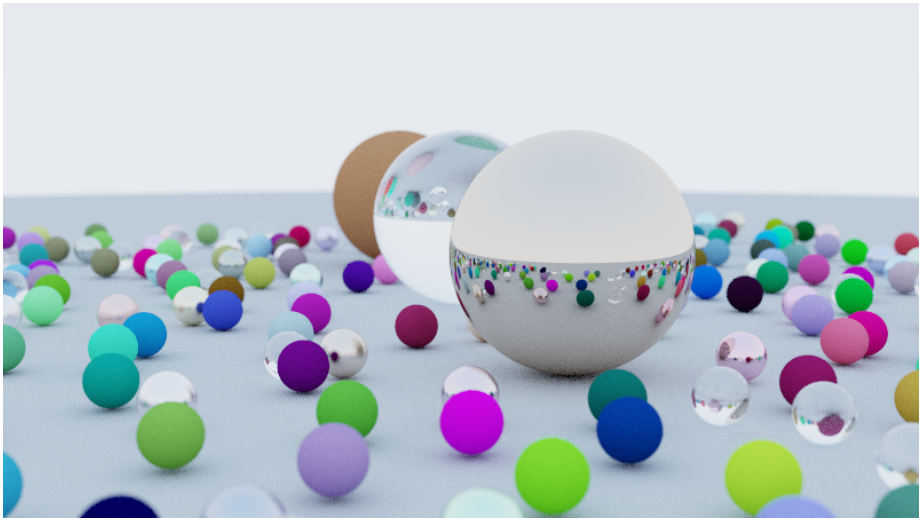


Figure 3: Output of Scene 1 (up to 200 spp at 800×450, adaptive): random spheres with depth of field, glass refraction, metal reflections and a sky-lit ground plane.

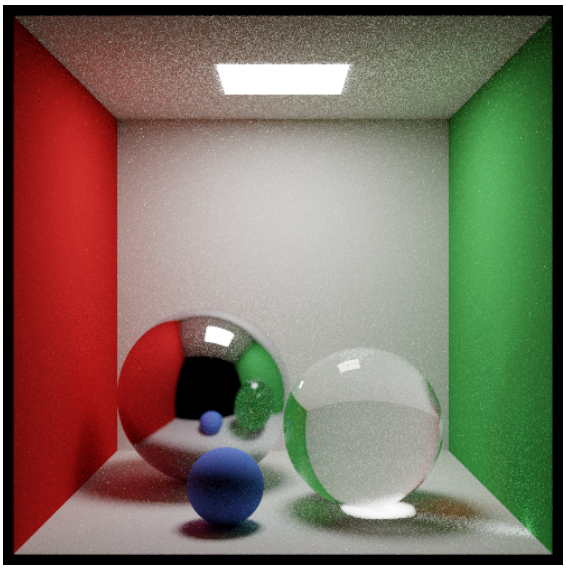


Figure 4: Output of Scene 2, the Cornell box (500×500, up to 400 spp, adaptive): colour bleeding from the red and green walls, soft shadows from the area light, a refractive glass sphere with a caustic on the floor, and a reflective metal sphere.

Observed global-illumination effects. In Figure 4 the white walls and floor take on a red and green tint near the side walls (*colour bleeding*); shadows under the spheres have soft penumbrae (*area-light soft shadows*); the glass sphere inverts and bends the scene behind it and focuses light into a bright patch on the floor (*caustic*); and the metal sphere mirrors the room including both coloured walls (*specular inter-reflection*). None of these effects is coded explicitly; they emerge from the path tracing loop.

8 Experiments and Performance Results

8.1 Build and Run

Listing 6: Build and run commands.

```
g++ -std=c++11 -O3 -pthread raytracer.cpp -o raytracer
./raytracer 2 500 400 1 1 0.03    # Cornell box, 500 px, 400 spp, 1 thread, BVH on
./raytracer 1 800 200 1 1 0.03    # random spheres, 800 px, 200 spp
```

Arguments are `scene width max_spp threads use_bvh noise_threshold`.

8.2 Console Output

The program reports the scene size, BVH statistics, render time, average samples per pixel and throughput. The following is the *unedited* output of the Cornell box run that produced Figure 4:

```
15 objects, 500x500, up to 400 spp, 1 threads, BVH on
BVH: 15 nodes, built in 0.0 ms
Render time: 82.44 s
Average samples/pixel: 222.7 (budget 400)
Rays traced: 396.0 M  (4.8 Mrays/s)
Saved output.png
```

and of the random-sphere scene (Figure 3):

```
278 objects, 800x450, up to 200 spp, 1 threads, BVH on
BVH: 327 nodes, built in 0.1 ms
Render time: 21.62 s
Average samples/pixel: 75.8 (budget 200)
Rays traced: 77.9 M  (3.6 Mrays/s)
Saved output.png
```

Test platform. All measurements were taken on a Linux (Ubuntu 24) virtual machine exposing a *single* CPU core, compiled with `g++ -O3`. Absolute throughput will be higher on a multi-core desktop; the relative comparisons below remain valid.

8.3 BVH Acceleration

Each scene was rendered twice with identical settings and random seeds, with the BVH enabled and disabled (brute-force loop over all primitives). Because the seeds are identical, both runs trace exactly the same number of rays, so the difference is purely intersection cost.

Table 4: Effect of the BVH on render time (1 thread, measured).

Scene	Prims	Rays (M)	No BVH (s)	BVH (s)	Mrays/s (BVH)	Speed-up
1: Spheres, 400×225, 32 spp	278	5.2	7.96	1.50	3.4	**5.3**×
2: Cornell, 200×200, 32 spp	15	8.3	2.56	1.81	4.6	**1.4**×

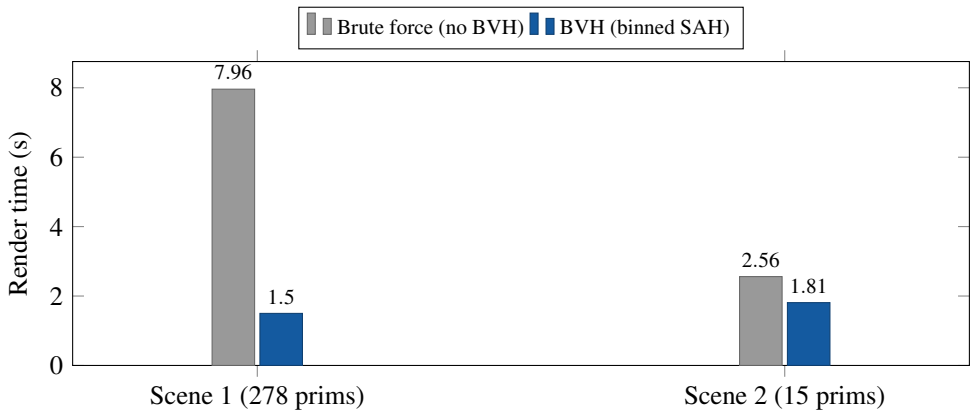
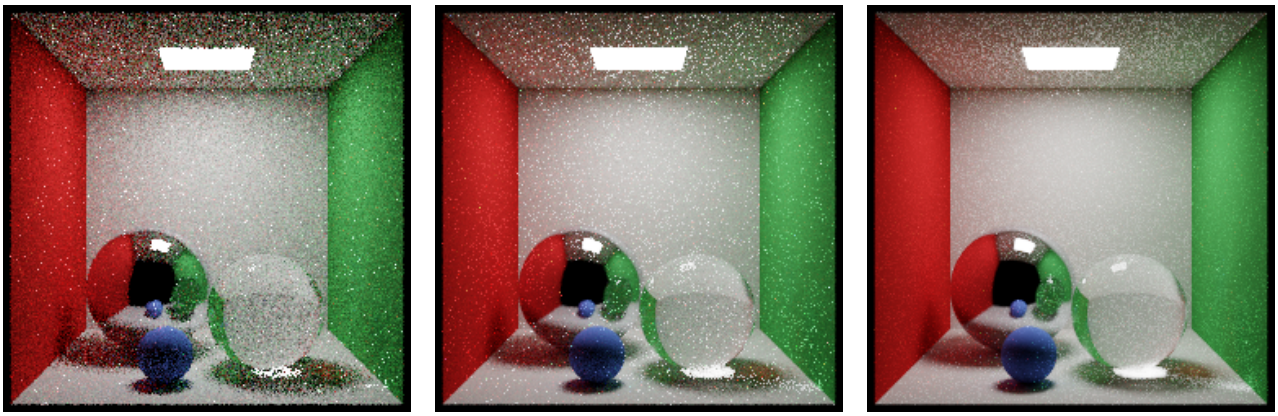


Figure 5: Measured render time with and without the BVH.

The benefit grows with scene complexity, exactly as the $O(N)$ versus $O(\log N)$ analysis predicts. On the 278-primitive scene the BVH cuts render time from 7.96 s to 1.50 s (**5.3**×) and raises throughput from 0.6 to 3.4 Mrays/s. On the 15-primitive Cornell box the gain is a modest 1.4×, because brute force over 15 primitives is already cheap and a BVH only pays off as the primitive count grows; scenes with tens of thousands of triangles would show far larger factors.

8.4 Sample Count and Convergence

Figure 6 shows the Cornell box at a fixed 4, 32 and 256 samples per pixel (adaptive sampling disabled with threshold 0). Statistical noise falls as $1/\sqrt{N}$, so going from 4 to 256 spp (64× more samples) should reduce the standard deviation about 8×; the broad grain on the walls and floor of the 4 spp image is visibly much finer at 256 spp, at the cost of a proportionally longer render (0.45 s to 22 s). The bright isolated dots that remain even at 256 spp are *fireflies*: rare, high-energy paths that reach the lamp only after bouncing off a diffuse wall and then through the glass sphere. Light sampling cannot help there, because the shadow ray is blocked by the (refractive) glass. These are the hardest paths for a unidirectional path tracer; the renderer clamps their contribution (Section 6), and Section 10 lists techniques that would remove them.

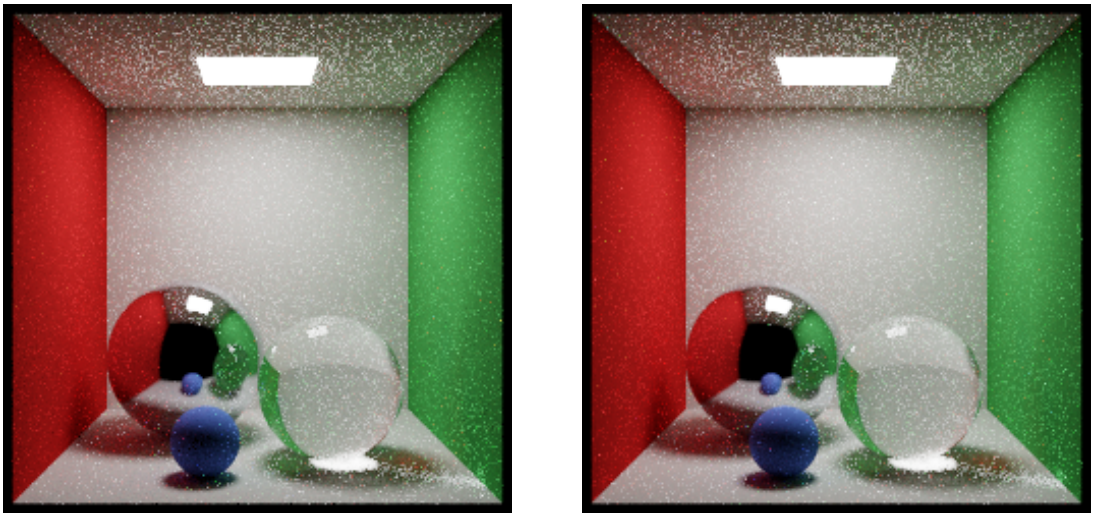


(a) 4 spp: 0.45 s **(b)** 32 spp: 2.86 s **(c)** 256 spp: 22.08 s

Figure 6: Convergence of the Cornell box at 250×250 pixels with increasing samples per pixel.

8.5 Adaptive Sampling

Both renders in Figure 7 use a 128 spp budget. The left image uses the default noise threshold of 0.03, the right image forces every pixel to use the full budget.



(a) Adaptive ($\tau = 0.03$): 10.06 s, avg 103.9 spp **(b)** Fixed ($\tau = 0$): 10.96 s, avg 128.0 spp

Figure 7: Adaptive versus fixed sampling at the same maximum budget (128 spp).

Table 5: Adaptive sampling statistics (Cornell box, 250×250, budget 128 spp).

Mode	Avg spp	Rays (M)	Render time (s)
Fixed ($\tau = 0$)	128.0	52.3	10.96
Adaptive ($\tau = 0.03$)	103.9	45.5	10.06

8.6 Multi-Threading

Table 6 lists renders of the same scene with 1, 2 and 4 worker threads. The three output files have the *identical* MD5 checksum (`efd6e0031204b7e0400a5f9495659706`), proving that the per-tile seeding makes the renderer deterministic and free of data races regardless of scheduling.

Table 6: Thread-count test (Cornell box, 200×200, 32 spp).

Threads	Time (s)	Rays (M)	Image MD5 (first 8 chars)
1	1.81	8.3	`efd6e003`
2	1.82	8.3	`efd6e003`
4	1.82	8.3	`efd6e003`

The test machine exposes only one core, so the wall-clock time cannot fall as threads are added; the table therefore demonstrates *correctness* (identical image, identical ray count, negligible threading overhead) rather than speed-up. Because rendering is embarrassingly parallel, with independent tiles, no shared mutable state during rendering and a lock-free work queue, the expected speed-up on an n-core CPU is close to n (Amdahl's law with a serial fraction limited to scene set-up, BVH build of about 0.1 ms and PNG encoding). Run `./raytracer 2 800 400 <cores>` on a multi-core machine to measure it.

9 Testing and Validation

Table 7: Validation checks performed.

Check	Method / evidence	Result
Geometry correctness	Spheres and quads appear with correct perspective and no gaps at wall joints (Fig. 4)	Pass
Colour bleeding	Red/green tint on adjacent white surfaces	Pass
Soft shadows	Penumbrae under spheres from the area light	Pass
Refraction	Inverted image through glass sphere; Snell + total internal reflection	Pass
Reflection	Metal sphere mirrors both side walls	Pass
Depth of field	Scene 1: hero spheres sharp, small spheres blur with distance	Pass
BVH correctness	Same seeds with/without BVH trace identical ray counts (Table 4) and visually identical images	Pass
No self-shadowing	$\varepsilon = 10^{-3}$ offset; no acne on diffuse surfaces	Pass
Thread safety / determinism	Identical MD5 for 1, 2, 4 threads (Table 6)	Pass
Energy / noise convergence	Noise decreases with spp (Fig. 6)	Pass
Valid PNG output	File opens in viewers; used directly in this report	Pass

10 Limitations and Future Work

- **Primitive set.** Only spheres and triangles are supported, and scenes are defined in code. Adding an OBJ mesh loader would allow arbitrarily complex models, for which the BVH would be indispensable.
- **Polymorphic design.** Introducing an abstract `Hittable` / `Material` interface with virtual `hit()` and `scatter()` methods would make new primitives and materials pluggable. The BVH could keep a flat array of primitive indices to preserve cache efficiency.
- **Textures.** Image textures, procedural noise (Perlin) and bump mapping are not implemented.
- **Sampling.** Only light sampling is combined with BRDF sampling in a simple way; full Multiple Importance Sampling, quasi-random sequences, and a denoiser would reduce noise further. Caustics through glass remain noisy

and produce fireflies, which are clamped rather than solved.
- **Acceleration.** SIMD packet traversal, a flattened "wide" BVH and SAH-based spatial splits could raise throughput several-fold. Frame-level GPU offloading is also possible.
- **Output.** The PNG writer stores data uncompressed; adding DEFLATE compression would reduce file size.

11 Conclusion

The project delivered a complete, dependency-free C++ path tracer in a single source file that meets every requirement of the problem statement. Global illumination effects (colour bleeding, soft shadows, caustics, reflections and refractions) are produced by the Monte Carlo path tracing core; next-event estimation and adaptive sampling reduce noise and wasted work; a binned-SAH BVH accelerates intersection by 5.3× on the 278-primitive scene and its benefit grows with scene size; and a lock-free tile scheduler with per-tile seeding provides thread-count-independent, reproducible images. The object-oriented structure, built from small value types, operator overloading, composition, lambdas and STL containers, kept the code short (about 600 lines) and readable. The work offers a solid base for extensions such as mesh loading, textures and polymorphic materials.

References

[1] P. Shirley, T. D. Black, S. Hollasch, *Ray Tracing in One Weekend* series (*In One Weekend*, *The Next Week*, *The Rest of Your Life*). https://raytracing.github.io

[2] M. Pharr, W. Jakob, G. Humphreys, *Physically Based Rendering: From Theory to Implementation*, 4th ed., MIT Press, 2023.

[3] J. T. Kajiya, "The Rendering Equation," *ACM SIGGRAPH Computer Graphics*, 20(4), 1986.

[4] T. Möller and B. Trumbore, "Fast, Minimum Storage Ray/Triangle Intersection," *Journal of Graphics Tools*, 2(1), 1997.

[5] J. D. MacDonald and K. S. Booth, "Heuristics for Ray Tracing Using Space Subdivision," *The Visual Computer*, 6, 1990.

[6] C. Schlick, "An Inexpensive BRDF Model for Physically-based Rendering," *Computer Graphics Forum*, 13(3), 1994.

[7] K. Narkowicz, "ACES Filmic Tone Mapping Curve," 2016.

[8] T. Boutell et al., *PNG (Portable Network Graphics) Specification*, W3C Recommendation, 2003.

A Command-Line Interface

Table 8: Program arguments (all optional, positional).

#	Argument	Default	Meaning
1	scene	1	1 = random spheres, 2 = Cornell box
2	width	800	Image width in pixels (height from aspect ratio)
3	max_spp	200	Maximum samples per pixel
4	threads	all cores	Number of worker threads
5	use_bvh	1	1 = BVH, 0 = brute force
6	noise_threshold	0.03	Adaptive sampling tolerance (0 = fixed spp)